Edge cases (when the answer spans two chunks) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py` (Edge cases section): you build this exact failure on purpose, here, where nothing's at stake — so the first time it happens for real, in the Build Task's knowledge base, you recognize it immediately instead of trusting a confident, half-right answer. **If not:** "the model gave a plausible but incomplete answer" would look identical to "the model doesn't know," and you'd have no idea which one you were debugging.

Basic Version

Approach 1 — see the split, and the k=1 vs k=2 gap

```
# chunking_practice.py — Edge cases section
import math
from openai import OpenAI

client = OpenAI()

document = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
chunk_size = 80

chunks = []
for i in range(0, len(document), chunk_size):
    chunks.append(document[i:i + chunk_size])

print("chunks:")
for i, chunk in enumerate(chunks):
    print(i, ":", repr(chunk))

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
```

```
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

chunk_embeddings = []
for chunk in chunks:
    chunk_embeddings.append(get_embedding(chunk))

question = (
    "What time and room is the quarterly meeting, and who is expected "
    "to attend?"
)
question_embedding = get_embedding(question)

scored = []
for i in range(len(chunks)):
    score = cosine_similarity(question_embedding, chunk_embeddings[i])
    scored.append((score, i))
scored.sort(reverse=True)

print()
print("search with k=1:")
for score, i in scored[:1]:
    print(chunks[i])

print()
print("search with k=2 (all chunks):")
for score, i in scored[:2]:
    print(chunks[i])
```

Expected output (first part):

```
chunks:
0 : 'The quarterly meeting will be held at 3 PM on Friday in Room 204,
     and everyone f'
1 : 'rom the product and engineering teams is expected to attend
     without exception.'
```

(shown wrapped onto two lines each just to fit the page — really one line of output each)

The boundary lands mid-word, mid-fact: chunk 0 has the time and room, chunk 1 has who is expected to attend. With `k=1`, the search returns only one of those two chunks — you get either the time/room or the attendee list, never both. With `k=2`, both chunks come back and together they contain the full answer.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a reusable `search(..., k)` function

```
# chunking_practice.py — Edge cases section
import math
from openai import OpenAI

client = OpenAI()

DOCUMENT: str = (
    "The quarterly meeting will be held at 3 PM on Friday in Room 204, "
    "and everyone from the product and engineering teams is expected "
    "to attend without exception."
)
CHUNK_SIZE: int = 80
QUESTION: str = (
    "What time and room is the quarterly meeting, and who is expected "
    "to attend?"
)


def chunk_text(text: str, chunk_size: int) -> list[str]:
    results = []
    for i in range(0, len(text), chunk_size):
        results.append(text[i:i + chunk_size])
    return results


def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding


def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
```

```
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)


def search(
    question: str,
    chunks: list[str],
    chunk_embeddings: list[list[float]],
    k: int,
) -> list[str]:
    question_embedding = get_embedding(question)
    scored = []
    for chunk, embedding in zip(chunks, chunk_embeddings):
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk))
    # how: tuples sort by their first item — the score — highest first
    scored.sort(reverse=True)
    top_chunks = []
    for score, chunk in scored[:k]:
        top_chunks.append(chunk)
    return top_chunks


def main() -> None:
    chunks = chunk_text(DOCUMENT, CHUNK_SIZE)
    print(f"document was cut into {len(chunks)} chunks:")
    for i, chunk in enumerate(chunks):
        print(f"  {i}: {chunk!r}")

    chunk_embeddings = []
    for chunk in chunks:
        chunk_embeddings.append(get_embedding(chunk))

    top_1 = search(QUESTION, chunks, chunk_embeddings, k=1)
    top_all = search(QUESTION, chunks, chunk_embeddings, k=len(chunks))

    print("\nk=1 result:")
    for chunk in top_1:
        print(" ", chunk)
    print(f"\nk={len(chunks)} result (all chunks):")
    for chunk in top_all:
```

```
        print(" ", chunk)


if __name__ == "__main__":
    main()
```

Difference from Basic: `search` is a reusable function with an explicit `k`, instead of inline tuple-sorting written out by hand — the shape real retrieval code needs, since a real pipeline calls this with different questions and different `k` values repeatedly. `k=len(chunks)` instead of a hardcoded `2` means the same code keeps working if `CHUNK_SIZE` changes and the document ends up in 3 or 4 chunks instead of 2.

Approach 2 — overlap removes the need for a bigger `k` at all

Story: a bigger `k` isn't actually a fix, it's a workaround that only happens to work in this 2-chunk toy example — the real fix is not splitting the fact in the first place. **If not:** the Build Task's real knowledge base would need an ever-larger `k` to keep papering over the same class of bug, feeding more irrelevant chunks to the model each time (this document's "lost in the middle" problem).

The rule for overlap: each new chunk starts `chunk_size - overlap` characters after the last one, so any piece of text up to `overlap` characters long always fits whole inside at least one chunk. Here the fact runs from `"3 PM"` to `"attend"` — 101 characters. That's longer than the old `chunk_size` of 80, so **no** overlap could save it at that size. The chunk must be bigger than the fact, and the overlap at least as long as it: `chunk_size=140, overlap=105`.

```
# chunking_practice.py — Edge cases section
def chunk_with_overlap(text: str, chunk_size: int, overlap: int) -> list[str]:
    chunks: list[str] = []
    # why: smaller than chunk_size -- chunks overlap
    step = chunk_size - overlap
    i = 0
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
        i += step
    return chunks


def main() -> None:
    overlap_chunks = chunk_with_overlap(DOCUMENT, chunk_size=140, overlap=105)
    print(f"overlapping chunks ({len(overlap_chunks)}):")
    for i, chunk in enumerate(overlap_chunks):
        whole_fact = "3 PM" in chunk and "attend" in chunk
        print(f"  {i}: whole fact inside = {whole_fact}")

    overlap_embeddings = []
```

```
    for chunk in overlap_chunks:
        overlap_embeddings.append(get_embedding(chunk))
    top_1_overlap = search(QUESTION, overlap_chunks, overlap_embeddings, k=1)

    print("\nk=1 result, WITH overlap:")
    for chunk in top_1_overlap:
        print(" ", chunk)


if __name__ == "__main__":
    main()
```

Expected output (the last line is wrapped here to fit the page):

```
overlapping chunks (5):
  0: whole fact inside = True
  1: whole fact inside = True
  2: whole fact inside = False
  3: whole fact inside = False
  4: whole fact inside = False

k=1 result, WITH overlap:
  The quarterly meeting will be held at 3 PM on Friday in Room 204, and
  everyone from the product and engineering teams is expected to attend
```

Chunks 0 and 1 each hold the time, the room *and* who must attend. Chunks 2–4 are short leftovers from the end of the document. The question matches the whole-fact chunks best, so `k=1` now returns the complete answer — something `k=1` against the non-overlapping 80-character chunks could never do. The cost: 5 chunks instead of 2, with lots of repeated text.

Approach 3 — proving `k` alone never fully solves this, at any size

Story: Approach 2's fix works, but it's still worth seeing *why* raising `k` was never a real guarantee — burying the same fact inside a much longer document shows a bigger `k` still competing for a fixed number of slots against dozens of other chunks. **If not:** "just raise `k`" would keep looking like a real fix until it quietly stopped working on a document bigger than this exercise's toy example.

```
# chunking_practice.py — Edge cases section
# same fact, but now buried inside a much longer document with lots of
# other, unrelated sentences before and after it
padding = "This is unrelated filler text about the cafeteria menu. " * 20
long_document = padding + DOCUMENT + padding

chunks = chunk_text(long_document, chunk_size=80)
print(f"long document produced {len(chunks)} non-overlapping chunks")
```

```
chunk_embeddings = []
for chunk in chunks:
    chunk_embeddings.append(get_embedding(chunk))
top_3 = search(QUESTION, chunks, chunk_embeddings, k=3)

print("k=3 result on the long document:")
for chunk in top_3:
    print(" ", chunk)
```

Expected output (illustrative): with dozens of chunks now competing for the top spots, `k=3` is no longer guaranteed to include *both* halves of the split fact — the "unrelated filler" chunks don't win, but the two fact-halves might not both make the top 3 either, depending on exactly how the cut fell relative to the surrounding filler text. This is the point: raising `k` was never a guarantee, just better odds in a small example. Chunking with overlap (Approach 2) removes the problem at its source — as long as the overlap is at least as long as the fact — regardless of how many other chunks exist.

Difference from Approach 1, and between Approaches 2/3: Approach 1 shows that a bigger `k` recovers the full fact *in this specific 2-chunk document*. Approach 2 fixes the actual cause — with an overlap at least as long as the fact, the fact always sits whole inside some chunk, so even `k=1` gets the whole thing. Approach 3 demonstrates why that matters: once the document is realistically sized, a bigger `k` stops being a reliable fix at all, because it's competing against many more chunks for a fixed number of "winning" slots.

Which one should you actually write? Overlap (Approach 2) — always, for any fixed-size chunker used on real documents, with the overlap at least as long as the facts you need kept whole (a sentence or two). It costs a little redundant storage (each chunk shares text with its neighbors) in exchange for removing an entire class of "half the fact went missing" bugs. Treat a bigger `k` as a mitigation you might still use for other reasons (recall on genuinely separate but related chunks), never as the actual fix for a boundary-split fact — Approach 3 is the proof that raising `k` doesn't scale the way it might feel like it does in a 2-chunk toy example.